COMP0240 Coursework 1

Structural Inspection Path Planning Challenge

Bowen Zhu | March 2026

1. Problem Statement and Objectives

Autonomous drone-based structural inspection requires visiting a set of camera viewpoints to photograph markers on structures, while navigating around obstacles in the environment. This mirrors real-world inspection scenarios where drones must efficiently survey buildings, bridges, or infrastructure while maintaining safe clearance from the structure and surrounding obstacles.

The challenge decomposes into two coupled optimisation problems: (1) **tour optimisation** -- determining the most efficient visitation order across all viewpoints to minimise total flight time and distance; and (2) **collision-free motion planning** -- routing the drone between consecutive viewpoints while avoiding cuboid obstacles. Four scenarios of increasing complexity evaluate the system:

Scenario	Viewpoints	Obstacles	Key Challenge
1	10	1	Basic route optimisation
2	10	5	Moderate obstacle density
3	20	1	Scalability to large viewpoint sets
4	5	10	Dense obstacle field navigation

The objectives are to: achieve 100% viewpoint visitation across all scenarios, minimise total mission time and flight distance compared to a sequential baseline, avoid all obstacle collisions, and verify task completion by capturing photographs at each viewpoint showing the ArUco markers.

2. Methodology

The solution decouples the problem into two stages: a global route optimiser determines the visitation order, then a local path planner handles obstacle avoidance for each segment. This decomposition simplifies implementation while achieving strong performance, as the TSP and motion planning sub-problems can each be solved with well-established algorithms.

2.1 Tour Optimisation (TSP)

The viewpoint visitation order is formulated as an open-loop Travelling Salesman Problem (TSP). Starting from the drone's initial position, the solver finds the shortest tour visiting all viewpoints exactly once. The 3D Euclidean distance between all position pairs forms the cost matrix, with the return-to-start column zeroed since the drone lands at the final viewpoint rather than returning to its origin.

A **hybrid solver** was chosen to balance optimality with computational feasibility:

- **Exact dynamic programming** ($O(n^2 * 2^n)$) for scenarios with $n \leq 15$ viewpoints (Scenarios 1, 2, and 4), guaranteeing the globally optimal tour. With n=10, this computes in under 1 second.
- **Simulated annealing** for Scenario 3 (n=20), where DP would require $20^2 * 2^{20} \approx 419$ million states. SA provides near-optimal solutions through stochastic neighbourhood search with temperature-based acceptance of worse solutions to escape local minima.

The `python_tsp` library provides both solvers. This hybrid approach was chosen over pure heuristics (which sacrifice optimality for small n) or pure exact methods (which become infeasible for n > 15).

2.2 Collision-Free Motion Planning (3D A*)

Before executing each flight segment, a collision check samples points along the straight-line path at 0.2m intervals. If any sample falls within an obstacle's bounding box (expanded by a 1.0m safety padding to account for drone size and positional uncertainty), the path planner activates.

A* was chosen over alternatives for the following reasons: unlike Dijkstra's algorithm, A*'s heuristic (Euclidean distance to goal) significantly reduces search space in sparse 3D environments; unlike RRT, A* produces deterministic, optimal paths on the discretised grid; and unlike Dubins paths, A* naturally handles 3D obstacle avoidance without requiring separate 2D planning and altitude management.

The avoidance strategy uses a **three-tier approach** to prioritise computational efficiency:

1. **Elevation bypass:** Attempts to fly over obstacles by raising altitude incrementally (+2m, +3m, +4m, +5m above the higher endpoint), verifying that ascent, traverse, and descent segments are all collision-free. This requires only 3 waypoints and avoids the computational cost of grid search.
2. **Full 3D A* search:** If elevation bypass fails (e.g., very tall obstacles), a grid-based search (resolution 0.5m, 26-connected neighbourhood) finds a path through 3D space. Path simplification iteratively removes intermediate waypoints where direct line-of-sight exists, reducing flight segments.
3. **High-altitude fallback:** As a last resort, the drone ascends to 9.0-9.5m to overfly all obstacles.

2.3 Integration with Gazebo Simulation

The solution integrates with AeroStack2's ROS2 interface through the `DroneInterface` API. For each segment, the drone flies to intermediate avoidance waypoints using `go_to_point` (maintaining current yaw), then upon reaching the final viewpoint, executes `go_to_point_with_yaw` to rotate in-place to the specified camera orientation before photographing the marker.

A critical implementation detail concerns the yaw angle convention. The scenario files specify yaw in radians, and the AeroStack2 Python API documentation for `go_to_point_with_yaw` states the angle parameter is in degrees. However, inspection of the underlying ROS2 message definition (`YawMode.msg`) reveals the field is defined as `float32 angle # Fixed yaw`

`(rad)`, and the Python layer performs no conversion. Therefore, **radians must be passed directly** despite the misleading docstring. This discovery resolved initial marker alignment issues where the camera was not accurately facing the ArUco targets.

At each viewpoint, the drone pauses for 1 second after rotating to allow camera stabilisation, then captures a screenshot via ROS2 camera topic subscription. These screenshots serve as proof of task completion, showing the ArUco markers at each inspection point. All metrics (time, distance, visit order, avoidance details, intermediate waypoints) are logged to JSON files for post-flight analysis and visualisation.

3. Results and Discussion

3.1 Performance Summary

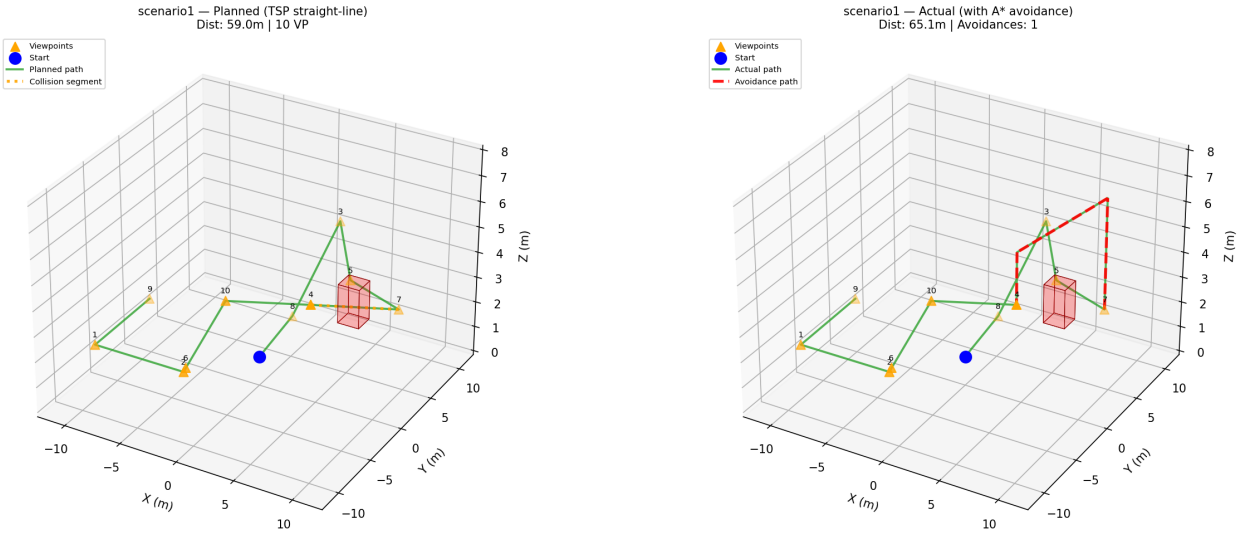
Scenario	Viewpoints	Obstacles	Time (s)	Distance (m)	Avoidances	Success
1	10/10	1	126.37	59.00	1	100%
2	10/10	5	143.23	59.00	1	100%
3	20/20	1	213.50	81.18	1	100%
4	5/5	10	112.95	47.78	3	100%

Table 2: Optimised mission performance across all four scenarios.

All scenarios achieve **100% viewpoint visitation** with zero collisions. The TSP solver produces identical optimal distances for Scenarios 1 and 2 (59.0m) despite different obstacle counts, confirming that the viewpoint layout -- not the obstacles -- determines tour length. Scenario 3 scales sub-linearly: doubling viewpoints from 10 to 20 increases distance by only 37% (59.0m to 81.2m), demonstrating the SA solver's effectiveness.

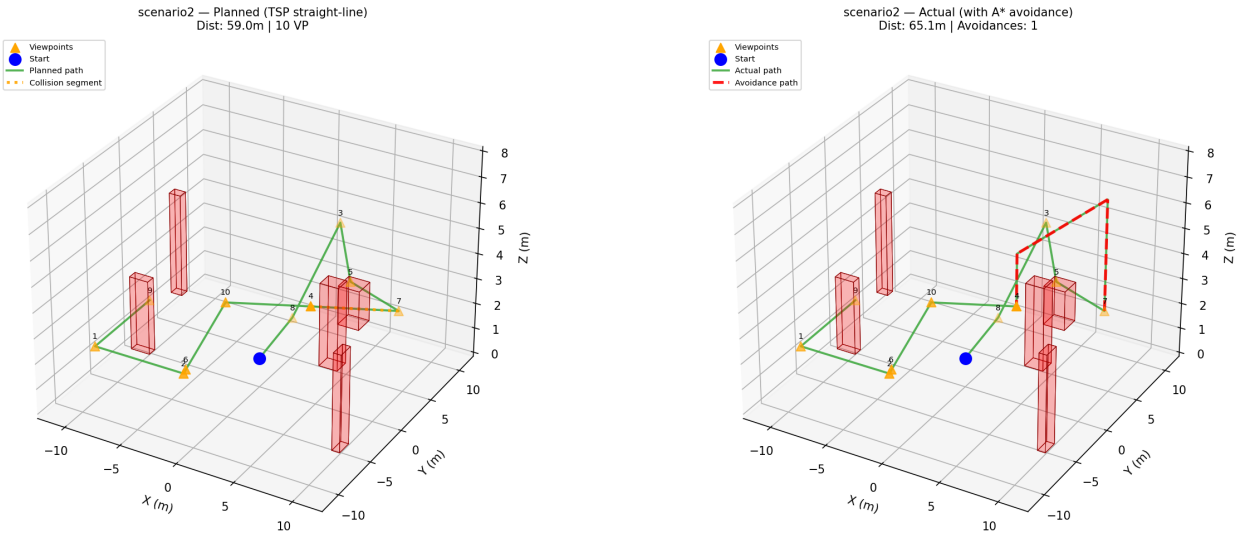
3.2 Planned vs Actual Flight Paths

Figures 1a-1d compare TSP-planned straight-line paths (left) against actual flight paths with A* obstacle avoidance (right). Orange dotted lines highlight segments where the planned path would collide with obstacles; red dashed lines show the elevation bypass trajectories actually flown.



Distance overhead from obstacle avoidance: 6.1m (+10.4%)

Figure 1a: Scenario 1 (10 VP, 1 obstacle) -- planned vs actual flight path.



Distance overhead from obstacle avoidance: 6.1m (+10.4%)

Figure 1b: Scenario 2 (10 VP, 5 obstacles) -- planned vs actual flight path.

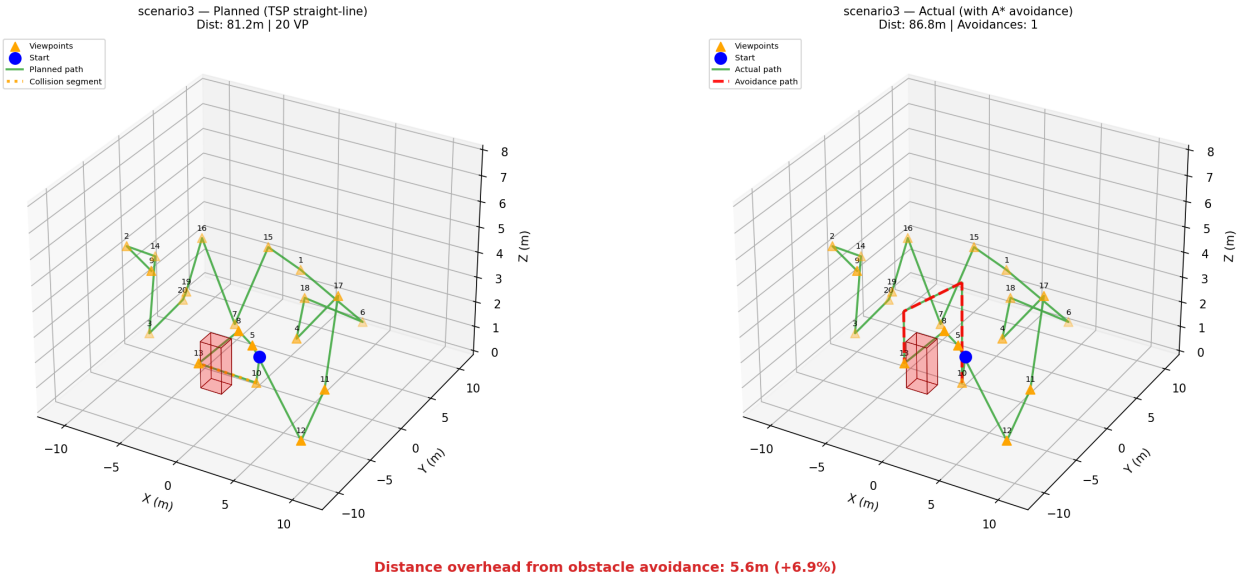


Figure 1c: Scenario 3 (20 VP, 1 obstacle) -- planned vs actual flight path.

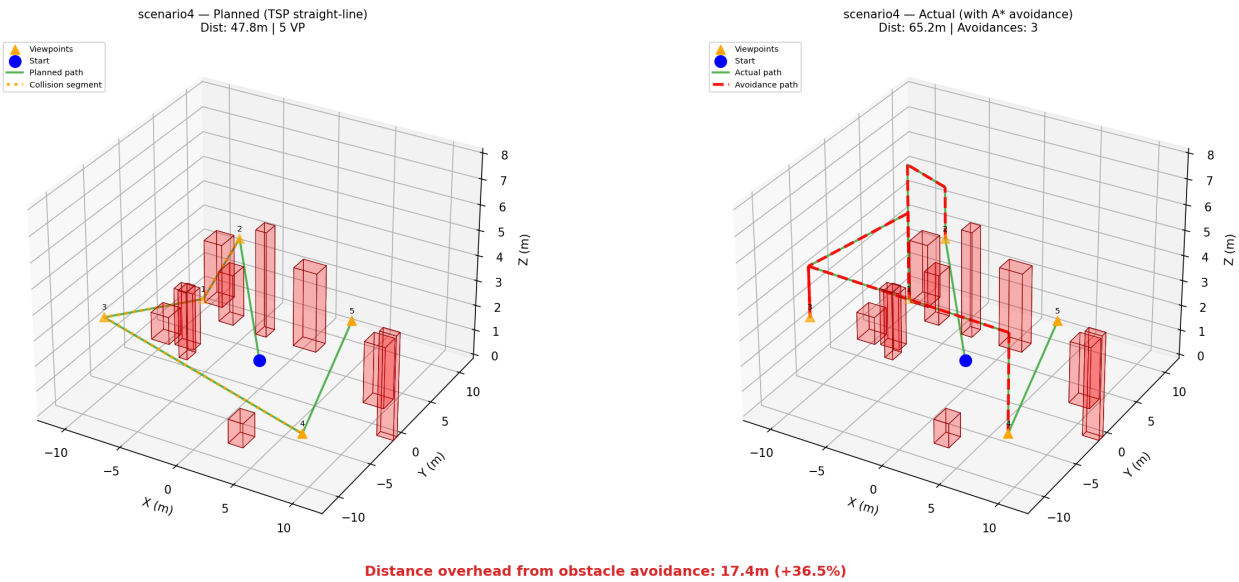


Figure 1d: Scenario 4 (5 VP, 10 obstacles) -- planned vs actual flight path.

3.3 Obstacle Avoidance Overhead

Scenario	Planned (m)	Actual (m)	Overhead	Strategy
1	59.0	65.1	**+6.1m (+10.4%)**	1 elevation bypass
2	59.0	65.1	**+6.1m (+10.4%)**	1 elevation bypass
3	81.2	86.8	**+5.6m (+6.9%)**	1 elevation bypass
4	47.8	65.2	**+17.4m (+36.5%)**	3 elevation bypasses

Table 3: Distance overhead from obstacle avoidance.

The elevation bypass strategy resolved all obstacle conflicts across all scenarios -- neither full A* search nor the high-altitude fallback were required. In Scenario 4, the drone ascended to z ≈ 5-7m to overfly dense obstacle clusters, with the longest segment (VP3 to VP4) spanning the full

arena width. The +36.5% overhead in Scenario 4 highlights the main limitation: the TSP solver plans routes assuming straight-line distances and is unaware of obstacles, potentially routing through obstacle-dense areas.

Comparing Scenarios 1 and 2 is instructive: both have identical optimal tour distances (59.0m) and identical avoidance overhead (+6.1m), despite Scenario 2 having 5x more obstacles. This indicates that obstacle placement relative to the optimal tour matters more than raw obstacle count.

4. Conclusions

The combined TSP + 3D A* approach achieves 100% success across all four scenarios with zero collisions. The hybrid TSP solver (exact DP for $n \leq 15$, simulated annealing for larger sets) scales effectively from 5 to 20 viewpoints, and the three-tier obstacle avoidance strategy provides robust collision-free navigation with 6.9-36.5% distance overhead depending on obstacle density.

The primary limitation is the decoupled planning approach: the TSP solver uses straight-line distances that ignore obstacles, potentially producing tours that require costly detours. Future work could address this through obstacle-aware cost matrices that use actual avoidance distances, or iterative refinement where the TSP is re-solved after computing avoidance paths. Additionally, incorporating Dubins paths could better account for the drone's kinematic constraints, and real-time replanning could handle dynamic obstacles in real-world deployments.